Java Capstone Project:

A Java Capstone Project is a final project that combines all the Java concepts you have learned to develop a real-world application.

Example: Student Management System

Project objective: Develop an application to manage student details, marks, and records using Java and MySQL.

Technologies used

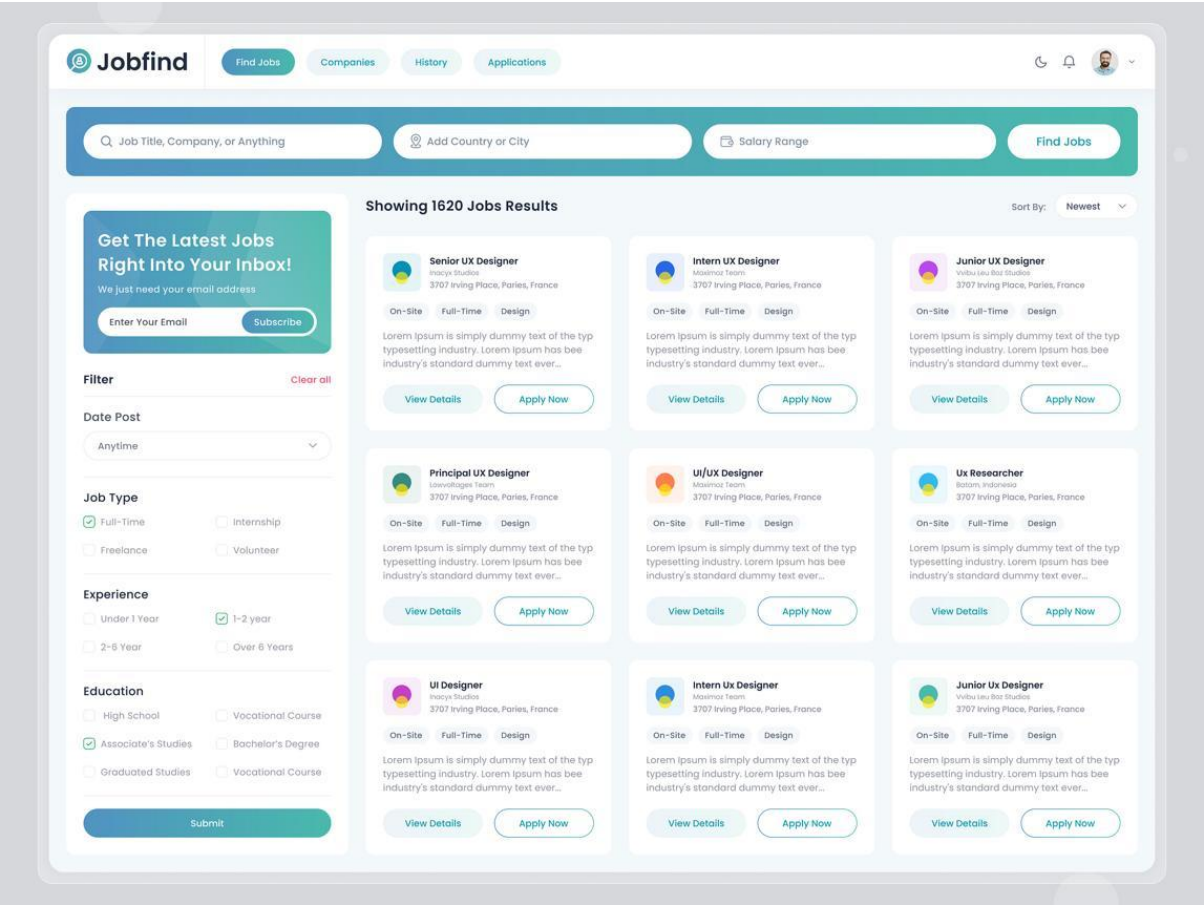
- Core Java: Classes, objects, OOP concepts, collections, and exception handling.
- JDBC: Connect Java with the MySQL database.
- MySQL: Store student information.
- Spring Boot: Develop REST APIs.
- Postman: Test REST API operations.
- HTML and CSS: Create a simple user interface (optional).

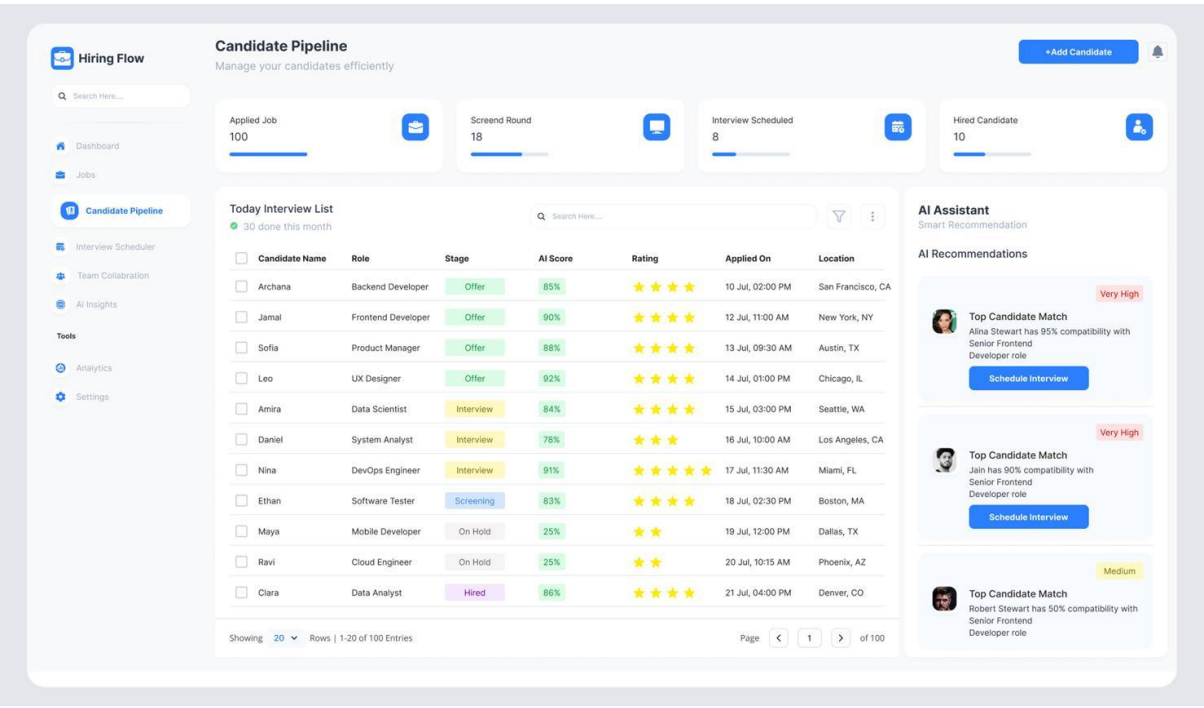
Design and build a complete Java/Spring Boot backend application (e.g. E-commerce Backend, Hospital Management, Job Portal):

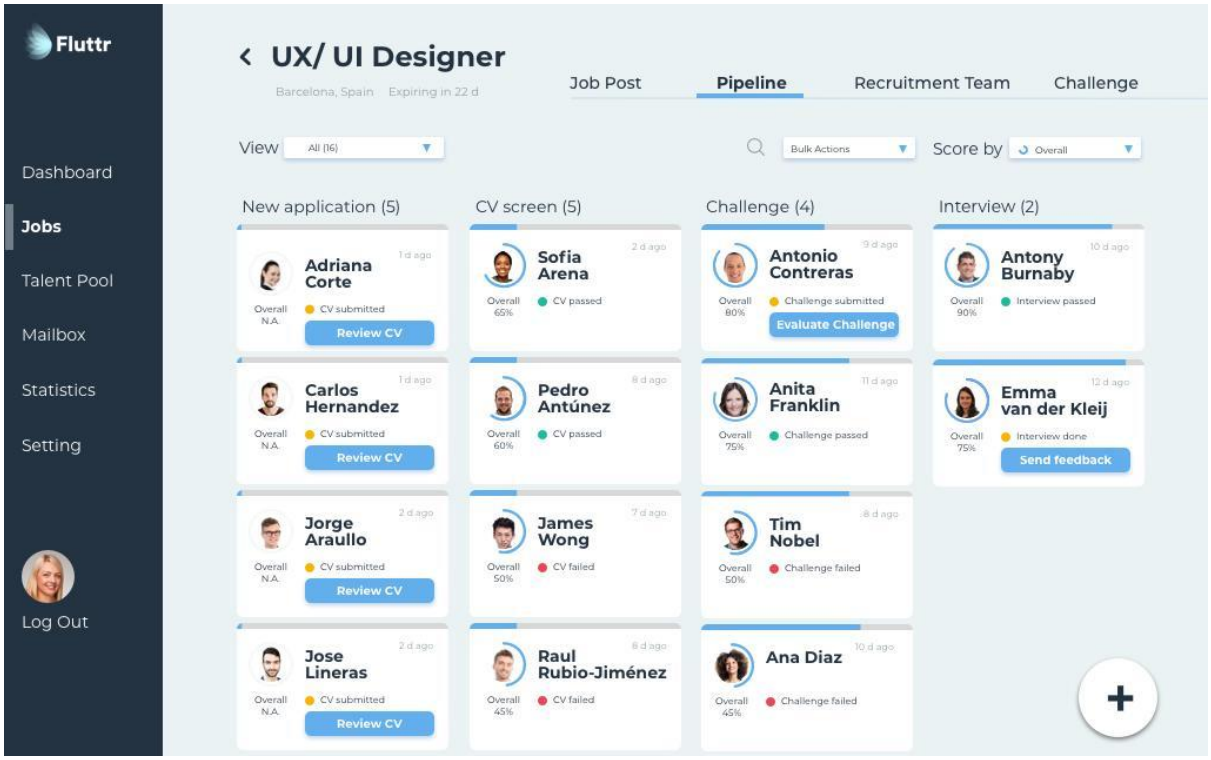
Java Capstone Project: Complete Backend Application

Simple definition: Design and build a complete Java/Spring Boot backend application means developing the server-side part of a real-world application. It handles business logic, database operations, user authentication, and REST APIs.

Example: Job Portal Backend


Jobfind
Find Jobs
Companies
History
Applications
Job Title, Company, or Anything
Add Country or City
Salary Range
Find Jobs
Get The Latest Jobs Right Into Your Inbox!
We just need your email address
Enter Your Email
Subscribe
Filter
Clear all
Date Post
Anytime
Job Type
Full-Time
Internship
Freelance
Volunteer
Experience
Under 1 Year
1-2 year
2-5 Year
Over 6 Years
Education
High School
Vocational Course
Associate's Studies
Bachelor's Degree
Graduated Studies
Vocational Course
Submit
Showing 1620 Jobs Results
Sort By:
Newest
Senior UX Designer
Intern UX Designer
Junior UX Designer
Principal UX Designer
UI/UX Designer
Ux Researcher
UI Designer
Intern Ux Designer
Junior Ux Designer
3707 Irving Place, Paries, France
On-Site
Full-Time
Design
Lorem Ipsum is simply dummy text of the typ typesetting industry. Lorem Ipsum has bee industry's standard dummy text ever...
View Details
Apply Now


Hiring Flow
Candidate Pipeline
Manage your candidates efficiently
+Add Candidate
Applied Job
100
Screend Round
18
Interview Scheduled
8
Hired Candidate
10
Today Interview List
30 done this month
Candidate Name
Role
Stage
AI Score
Rating
Applied On
Location
Archana
Backend Developer
Offer
85%
10 Jul, 02:00 PM
San Francisco, CA
Jamal
Frontend Developer
Offer
90%
12 Jul, 11:00 AM
New York, NY
Sofia
Product Manager
Offer
88%
13 Jul, 09:30 AM
Austin, TX
Leo
UX Designer
Offer
92%
14 Jul, 01:00 PM
Chicago, IL
Amira
Data Scientist
Interview
84%
15 Jul, 03:00 PM
Seattle, WA
Daniel
System Analyst
Interview
78%
16 Jul, 10:00 AM
Los Angeles, CA
Nina
DevOps Engineer
Interview
91%
17 Jul, 11:30 AM
Miami, FL
Ethan
Software Tester
Screening
83%
18 Jul, 02:30 PM
Boston, MA
Maya
Mobile Developer
On Hold
25%
19 Jul, 12:00 PM
Dallas, TX
Ravi
Cloud Engineer
On Hold
25%
20 Jul, 10:15 AM
Phoenix, AZ
Clara
Data Analyst
Hired
86%
21 Jul, 04:00 PM
Denver, CO
AI Assistant
Smart Recommendation
AI Recommendations
Very High
Top Candidate Match
Alina Stewart has 95% compatibility with Senior Frontend Developer role
Schedule Interview
Very High
Top Candidate Match
Jain has 90% compatibility with Senior Frontend Developer role
Schedule Interview
Medium
Top Candidate Match
Robert Stewart has 50% compatibility with Senior Frontend Developer role



12

A Job Portal Backend allows job seekers to register, search for jobs, and apply for jobs. Recruiters can post jobs and manage applications.

Technologies used

- Java: Implements the application logic.
- Spring Boot: Builds the backend application and REST APIs.
- Spring Data JPA / Hibernate: Connects Java objects to database tables.
- MySQL: Stores user, job, and application details.
- Spring Security: Protects APIs and manages authentication.
- Postman: Tests API requests and responses.

Main features

1. User registration and login.
2. Recruiters can post, update, and delete jobs.
3. Job seekers can view and search for jobs.
4. Candidates can apply for jobs.

5. Recruiters can view and update application statuses.
6. MySQL stores all application data securely.

Simple example: Spring Boot REST API

This example creates an API to return job details.

```
@RestController

@RequestMapping("/jobs")

public class JobController {

    @GetMapping

    public String getJobs() {

        return "Available Jobs: Java Developer, Python Developer";

    }

}
```

Output: When you send a GET request to http://localhost:8080/jobs in Postman, the API returns:

Available Jobs: Java Developer, Python Developer

How the backend works

Client / Postman

Sends an HTTP request

Spring Boot Controller

Receives the API request

Service Layer

Processes business logic

Repository + MySQL

Stores and retrieves data

Expected outcome

You will build a working backend application with CRUD operations (Create, Read, Update, Delete), database connectivity, REST APIs, input validation, and error handling.

Resume description: Developed a Job Portal Backend using Java, Spring Boot, REST APIs, Spring Data JPA, and MySQL to manage job postings, candidate profiles, and job applications.

Establish clean multi-tiered project architecture, databases, schemas, and API documentation:

1. Clean Multi-Tiered Project Architecture

Simple definition: Multi-tier architecture means dividing an application into separate layers, where each layer performs a specific task. This makes the project easy to develop, test, and maintain.

Example: Job Portal Backend

1. Controller Layer

Receives HTTP requests and returns responses.

Example: JobController.java

2. Service Layer

Contains business logic.

Example: JobService.java

3. Repository Layer

Communicates with the database.

Example: JobRepository.java

4. Database Layer

Stores and retrieves job details.

Example: MySQL

2. Database and Schemas

Simple definition: A database stores application information, while a schema defines the structure of the database, including tables, columns, data types, and relationships.

Example SQL for a Job Portal:

```
CREATE DATABASE jobportal;

USE jobportal;

CREATE TABLE jobs (
    id INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(100),
    company VARCHAR(100),
    location VARCHAR(100)
);

INSERT INTO jobs (title, company, location)
VALUES ('Java Developer', 'ABC Company', 'Hyderabad');


SELECT * FROM jobs;
```

Output:

id	title	company	location
1	Java Developer	ABC Company	Hyderabad

3. API Documentation

Simple definition: API documentation explains available APIs, their URLs, HTTP methods, required inputs, and expected responses. It helps developers understand and test the application.

Example API endpoints:

HTTP method	Endpoint	Purpose
GET	/jobs	View all jobs
GET	/jobs/1	View a specific job
POST	/jobs	Add a new job
PUT	/jobs/1	Update a job
DELETE	/jobs/1	Delete a job

In Spring Boot, you can use Swagger UI / OpenAPI to document and test APIs through a browser.

Final outcome

By completing this task, you will learn to:

- Organize Java code into Controller, Service, and Repository layers.
- Design MySQL tables and relationships.
- Connect Spring Boot to MySQL using Spring Data JPA.
- Create and document REST APIs using Swagger UI.
- Build a maintainable backend application suitable for your Java capstone project.

Document your software architecture, build processes, and database relations:

Document Software Architecture, Build Processes, and Database Relations

Simple definition: It means preparing clear documentation that explains how your software is organized, how to build and run the application, and how different database tables are connected.

Example: In a Java Spring Boot Job Portal Project, you can document these three areas.

1. Software Architecture

Software architecture explains how different components of your application work together.

Example:

Controller Layer

Receives API requests

Service Layer

Processes business logic

Repository Layer

Handles database operations

MySQL Database

Stores jobs and applications

2. Build Processes

The build process explains how to compile, test, package, and run your Spring Boot application.

Example commands using Maven:

Compile and test the project

mvn clean verify

Run the Spring Boot application

mvn spring-boot:run

The application typically runs at http://localhost:8080.

3. Database Relations

Database relations explain how tables are connected using primary keys and foreign keys.

Example for a Job Portal:

Users table

id — Primary Key

name, email

One user can submit many applications

Applications table

id — Primary Key

user_id — Foreign Key

job_id — Foreign Key

Many applications can refer to one job

Jobs table

id — Primary Key

title, company

For example, one candidate can apply for multiple jobs, and each job can receive applications from multiple candidates. The Applications table connects Users and Jobs.

What should your documentation contain?

- README.md — Project overview, technologies, setup, and run instructions.
- architecture.md — Application layers and component interactions.
- database-schema.sql — Table creation scripts and constraints.
- database-relations.md — Tables, primary keys, foreign keys, and relationships.
- API-documentation — API endpoints, request examples, and responses, preferably through Swagger/OpenAPI.

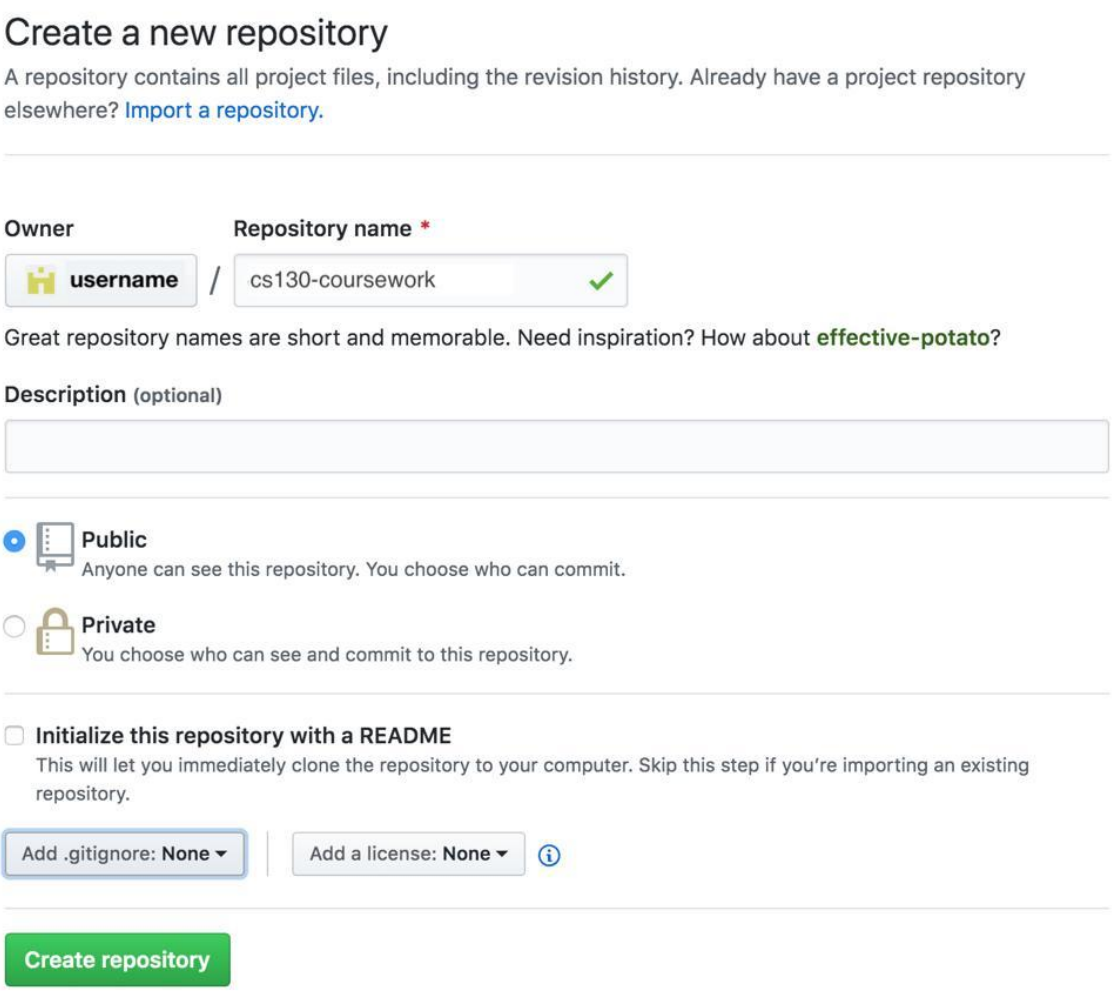
Final outcome: Your project becomes easier to understand, run, maintain, test, and present during technical interviews.

Create a GitHub repository and record a walkthrough demo video of the system features:

Create a GitHub Repository and Record a Walkthrough Demo Video

Simple definition: Upload your Java Capstone Project source code to GitHub so others can view it, and record a demo video explaining how the application works and demonstrating its main features.

1. Create a GitHub Repository



Steps to follow

1. Open GitHub and sign in.
2. Click New repository.
3. Enter a name, such as job-portal-backend.
4. Add a description and select Public or Private.
5. Click Create repository.
6. Upload your Java/Spring Boot project using Git or GitHub's upload feature.

Example repository structure:

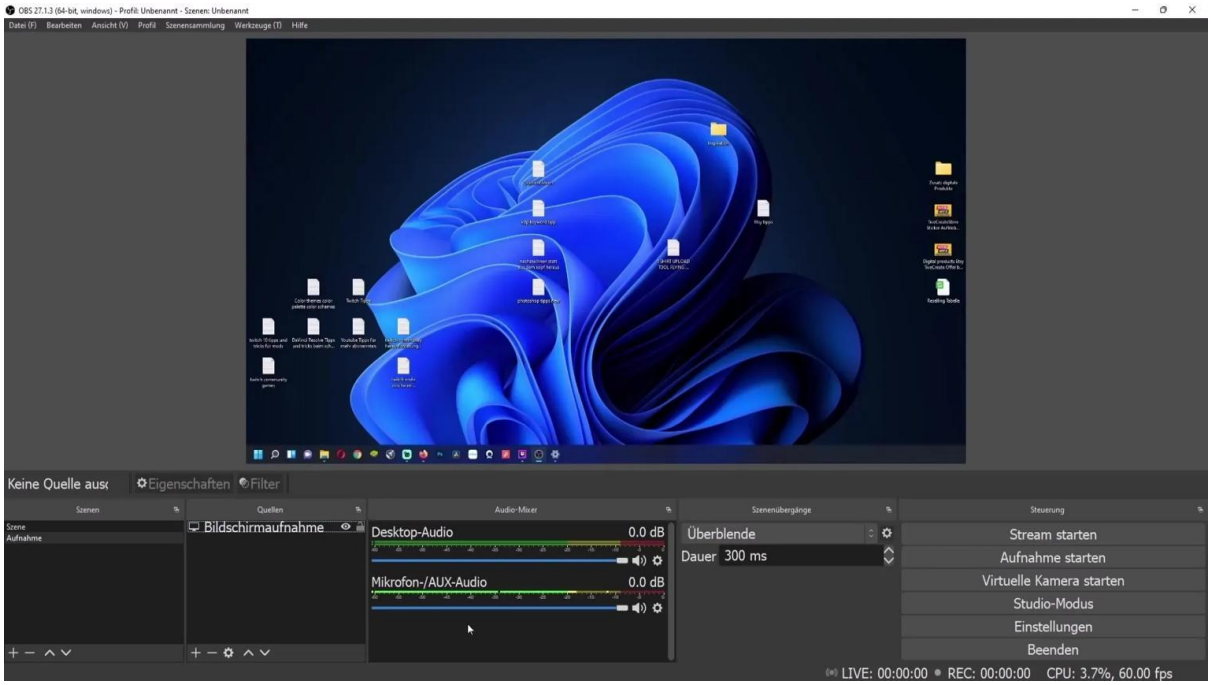
job-portal-backend/

├— src/

```
|   ├── main/java/
|   └── main/resources/
├── src/test/
├── pom.xml
├── README.md
├── database-schema.sql
└── .gitignore
```

Important: Never upload passwords, database credentials, API keys, or other secrets. Add them to environment variables or local configuration excluded by .gitignore.

2. Record a Walkthrough Demo Video



tutkit.com

You can use OBS Studio or your computer's built-in screen recorder.

Demo video steps:

1. Introduction (20 seconds): Explain your project name and purpose.
2. Architecture (30 seconds): Show the Controller, Service, Repository, and database layers.
3. Feature demonstration (1–2 minutes): Show job creation, viewing, updating, deleting, and application management.
4. API testing (30 seconds): Demonstrate successful requests and responses using Postman or Swagger UI.
5. Conclusion (10 seconds): Summarize the technologies and what you learned.

Aim for a clear, approximately 3-minute video.

3. Share Your Project

- GitHub: Share the repository URL on your resume and LinkedIn.
- Demo video: Upload the video to YouTube as Unlisted or share it through Google Drive.
- README: Include the project description, features, technologies, setup instructions, API endpoints, and demo video link.

Example resume entry

Job Portal Backend — Java Capstone Project

Developed a RESTful Job Portal Backend using Java, Spring Boot, MySQL, and Spring Data JPA, implementing CRUD operations, job application management, and API documentation. Uploaded the source code to GitHub and recorded a walkthrough demo showcasing the system's features.

Your next step: Create the repository first, upload your working code, and then record the demo using the actual running application.